

# The Test Suite

348 tests across 33 suites, and what they refuse to let the system say

---

EdgePilot AI - Team 15

Documentation set, 26 August 2026

6 documents in this volume

## tests/ - the Jest suite

**Work package:** shared, by subfolder, matching the module each suite covers.

### What this folder is

---

348 tests across 33 suites, mirroring `src/` one level down. Nothing here touches a network, a real timer, a real model or a real database - every adapter takes its `fetch` and its clock by injection, which is what makes the suite deterministic. A benchmark test that depended on real latency would be a flaky test that also proved nothing.

### What happened here

---

Eight files were added and six changed, taking the suite from 259 tests to 322. `tests/core/` is new; so are the suites for hardware fit, task compatibility, privacy assessment, comparison, comparison runs, the Ollama catalogue and the egress warning.

The framing is worth stating because it explains why the tests look the way they do. For a measurement tool the interesting question is not "does it work" but "**what does it refuse to say**". So most of these assert refusals: refuse to call a provider for a workload that does not exist, refuse to attribute a run to the wrong user, refuse to throw away a measurement because the database went down mid-run, refuse to declare a comparison winner inside the noise, refuse to turn a `null` into a `0`.

**What this suite cannot see.** Jest type-checks only what it imports. Two dev scripts and six schema-versus-interface mismatches were invisible to a green run and were caught by `next build`, which type-checks the whole project. That is why both are required checks.

One duplication is known and deliberate: `describeEgressWarning` is covered in both `egress-warning.test.ts` and `privacy-assessor.test.ts`. It is flagged rather than consolidated.

### Subfolders

---

| Subfolder                      | Covers                                                                        |
|--------------------------------|-------------------------------------------------------------------------------|
| <code>benchmark/</code>        | 19 files - the measurement core, the adapters, the routes                     |
| <code>core/</code>             | The session log                                                               |
| <code>dashboard/</code>        | The display helpers that enforce null-honesty                                 |
| <code>vision-benchmark/</code> | 5 files - the vision module end to end                                        |
| <code>evaluation/</code>       | The ten-case artefact's shape, including that no category can vanish silently |

### Connected folders

---

- `src/` - the tree this mirrors.
- `.github/workflows/` - runs `npm test` on every push.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- `src/modules/`

## tests/benchmark/ - the measurement core under test

**Work package:** benchmark and provider layer.

### What this folder is

Nineteen files covering the provider adapters, the runner, the use cases, the core services and the API route.

### What happened here

Seven files are new, all of them covering services that did not exist before: hardware fit, task compatibility, privacy assessment, comparison planning, comparison runs, the Ollama catalogue and the egress warning.

`run-benchmark.test.ts` changed most among the existing files (73 lines differing) - it uses the **real** `BenchmarkRunner` over scripted providers rather than a stubbed runner, so the envelope that comes out is the one the API actually returns, and the cold-start change moved through it.

One fix in here is worth calling out because of how it was made rather than what it was: a run of three iterations was reporting "4/4 iterations succeeded", because the success chain was counting the discarded warm-up. The fix went into the runner, not into the assertion.

### Files

| File                                       | What it covers                                                                                                                                                                   |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>helpers.ts</code>                    | The test doubles: fake providers, a stepped clock, a frozen clock, SSE and NDJSON response builders, connection-refused and hanging fetch stubs                                  |
| <code>benchmark-runner.test.ts</code>      | When the runner falls back. The rule: fall back only when the provider failed for a reason another provider could plausibly not share                                            |
| <code>run-benchmark.test.ts</code>         | The use case - ownership refusals, persistence failure mid-run, and the full envelope                                                                                            |
| <code>benchmarks-route.test.ts</code>      | Parse, dispatch, status code. Above all: refuse a malformed body <b>before</b> anything constructs a provider, because a provider call costs money, quota and up to five minutes |
| <code>benchmark-measurement.test.ts</code> | What the summary refuses to report. This is where a number would be invented if anywhere                                                                                         |
| <code>benchmark-config.test.ts</code>      | Environment validation and the server-side guard                                                                                                                                 |
| <code>hardware-fit.test.ts</code>          | VRAM residency, spill share, and the states between                                                                                                                              |
| <code>task-compatibility.test.ts</code>    | The asymmetric rule - vision can do text, text cannot do vision, embedding neither                                                                                               |
| <code>privacy-assessor.test.ts</code>      | Privacy class, disqualifiers, egress classification                                                                                                                              |
| <code>egress-warning.test.ts</code>        | The pre-flight warning, including that it does <b>not</b> carry the prompt text                                                                                                  |
| <code>comparison.test.ts</code>            | Planning and reporting: overlapping ranges must not produce a winner                                                                                                             |
| <code>run-comparison.test.ts</code>        | The comparison use case, including sequential execution for local entrants                                                                                                       |
| <code>visitor-keys.test.ts</code>          | Header parsing for a visitor's own cloud keys: trimmed, shape-checked, laid over the server's per provider                                                                       |
| <code>recorded-provider.test.ts</code>     | A browser-recorded run through the runner: cold start discarded, browser residency readings used, no cloud fallback; and the request schema's recording rules                    |
| <code>ollama-catalog.test.ts</code>        | Runtime status and the installed-model list                                                                                                                                      |
| <code>cloud-catalog.test.ts</code>         | Gemini and Groq model lists: filtering, pagination, key-in-header, and each failure message                                                                                      |
| <code>ollama-provider.test.ts</code>       | NDJSON streaming, nanosecond conversion, TTFT                                                                                                                                    |
| <code>gemini-provider.test.ts</code>       | SSE parsing, usageMetadata, header-carried key                                                                                                                                   |

| File                                   | What it covers                                                                                                        |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>groq-provider.test.ts</code>     | OpenAI-compatible SSE, <code>stream_options.include_usage</code> , <code>completion_time</code> in seconds            |
| <code>demo-provider.test.ts</code>     | The one promise that makes the demo adapter acceptable: nothing it produces can be mistaken for a measurement         |
| <code>provider-errors.test.ts</code>   | The published error vocabulary. This is what stops the code and <code>docs/local-model-setup.md</code> drifting apart |
| <code>provider-registry.test.ts</code> | Which adapters are usable in a given environment, and in what order                                                   |

## Connected folders

- `src/modules/benchmark/` - the module under test.
- `src/app/api/v1/benchmarks/` - the route `benchmarks-route.test.ts` drives with the container mocked.
- `docs/local-model-setup.md` - the error-code table `provider-errors.test.ts` pins.

## Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/`
- `src/app/api/v1/comparisons/`
- `src/app/api/v1/local-runtime/`
- `src/modules/benchmark/application/`
- `src/modules/benchmark/application/dtos/`
- `src/modules/benchmark/application/services/`
- `src/modules/benchmark/application/use-cases/`
- `src/modules/benchmark/core/`
- `src/modules/benchmark/core/ports/`
- `src/modules/benchmark/core/services/`
- `src/modules/benchmark/infrastructure/`
- `src/modules/benchmark/infrastructure/providers/`
- `tests/`

## tests/core/ - cross-cutting concerns under test

**Work package:** benchmark and provider layer.

### What this folder is

---

One suite, covering the session log.

### What happened here

---

New, alongside `src/core/logging/`. The tests exist mainly to pin the redaction rule, which is the part of the session log that would be easiest to relax by accident: prompt **content** is never recorded by default, only a length and a digest. A log that wrote out the same prompt the egress warning had just flagged would have defeated its own warning.

They also pin the opt-in behaviour: a request without a session header is not logged anonymously and not logged to a default bucket - it is not logged at all.

### Files

---

| File                                 | What it covers                                                                                                              |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>cloud-run-quota.test.ts</code> | The per-visitor cap on cloud vision runs made with the server's key: the allowance, the hour window, and session separation |
| <code>session-log.test.ts</code>     | 224 lines. Redaction, prompt digests, the share payload and its consent statement, and the in-memory store's bounds         |

### Connected folders

---

- `src/core/logging/` - the code under test.
- `src/app/api/v1/session-log/` - the endpoints that expose it.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- `src/core/`
- `src/core/quota/`
- `tests/`

## tests/dashboard/ - display helpers under test

**Work package:** product UI.

### What this folder is

---

One suite, and it defends one rule.

### What happened here

---

The wire contract makes every aggregate nullable on purpose, because reporting 0 ms for "we never got an answer" is the single most misleading number this system could produce. These tests pin the helpers that stop a null becoming a 0 on screen - which is the last place the rule could be broken, after the data layer has got it right.

### Files

---

| File           | What it covers                                                                                                                                      |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| format.test.ts | 77 lines. fmtMs, fmtNum, fmtPct, fmtElapsed, describeErrorCode, isUuid, isPlaceholderId - each asserted to render "not measured" rather than a zero |

---

### Connected folders

---

- src/components/dashboard/ - format.ts, the file under test.
- evidence/evaluation/ - case 9 of the ten-case matrix asserts the same rule one layer down, in the data.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- src/components/
- tests/

## tests/evaluation/ - the matrix artefact under test

**Work package:** benchmark and provider layer.

### What this folder is

---

One suite, pinning the shape of evidence/evaluation/ten-case-matrix.json.

### What happened here

---

New. The matrix is now rendered as a page, which means its shape is a contract rather than a private detail - and the failure it guards against is a quiet one: a category the page cannot render would otherwise **vanish from the list silently**, leaving a matrix that looks complete while showing nine cases.

That is the same class of defect as reporting 0 for "not measured": not a crash, just a result that is wrong in a way nobody notices.

### Files

---

| File                    | What it covers                                                                                                    |
|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| ten-case-matrix.test.ts | 134 lines. Case count, the six categories, and that every category the artefact declares can actually be rendered |

---

### Connected folders

---

- evidence/evaluation/ - the artefact.
- scripts/evaluation/ - what writes it.
- src/app/evaluation/ - the page that renders it.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- tests/

## tests/vision-benchmark/ - the vision module under test

**Work package:** vision benchmark.

### What this folder is

---

Five suites covering the vision workload from schema to provider.

### What happened here

---

vision-execution.test.ts changed most (86 lines differing), because the warm-up classification changed the shape of an execution.

That change is a useful record of how a small feature goes wrong. The warm-up consumed a deterministic test clock, so the timings moved - fixed by calling provider.classify() directly rather than the timed wrapper. It fired on cloud providers, which meant a billable call and one extra image sent to a third party - fixed by restricting it to local providers and adding an explicit warmUp?: boolean. And it ate the first of the scripted outputs.shift() responses in two existing tests - those two now opt out, with a comment saying why, and two new tests cover the warm-up behaviour properly.

vision-api.test.ts changed to accept datasets whose label set is not the built-in seven.

### Files

---

| File                          | What it covers                                                          |
|-------------------------------|-------------------------------------------------------------------------|
| vision-benchmark.test.ts      | 356 lines. The workload end to end: metrics, thresholds, evidence shape |
| vision-execution.test.ts      | 362 lines. The executor, including warm-up behaviour and its opt-out    |
| vision-providers.test.ts      | The Ollama, Gemini and Groq vision adapters                             |
| vision-infrastructure.test.ts | 204 lines. Image processing, manifest loading, the evidence store       |
| vision-api.test.ts            | 78 lines. The /api/v1/vision-benchmarks route                           |

### Connected folders

---

- src/modules/vision-benchmark/ - the module under test.
- datasets/vision-benchmark/ - the fixtures pretest regenerates before these run.
- src/app/api/v1/vision-benchmarks/ - the route.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- src/modules/vision-benchmark/application/
- src/modules/vision-benchmark/infrastructure/
- tests/